Colin Chu, Le Fan Fang

Indoor Scene Recognition Using CNNs and Bayesian Modeling

Introduction and Literature Review

Motivation:

Indoor scene recognition is essential for service systems that help with accessibility and safety, especially for the visually impaired/blind. Reliable GPS signals are often not available, so navigation systems on robots often rely on visual recognition. For a robot to perform a task, for example go to the closest chair in the living room), it must first recognize what part of its environment is the living room. Our goal is to reliably guide the visually impaired through unknown environments.

Evolution of Indoor Scene Recognition:

Early indoor scene recognition systems relied on manually implemented feature engineering. The field at the time was dominated by the Bag of Visual Words (BoVW) model, adapted from document classification. In this method, an image was treated as an orderless collection of local feature descriptors[1]. At this time, researchers were still developing a method to create a robust system against changes in illumination, scale, and rotation.

A major limitation of early BoVW models was the loss of spatial layout information. To address this, Spatial Pyramid Matching (SPM) was introduced, which preserved spatial information by separating the image into increasingly small sub-regions and then adding up all their local features. These early systems had very limited success, with the best models achieving only 25% accuracy on the MIT-67 dataset[1].

The development of deep CNNs marked a transition away from manual feature extraction to deep neural networks. The use of CNNs allowed the capturing of features that were too small or that were too complicated for manual engineering.

Indoor scene recognition also benefited from using huge datasets like **ImageNet** as well as **Places.** Researchers found out that models that were trained on these huge datasets can be fine tuned for indoor scene recognition, which finally broke the 80%+ threshold on the MIT-67 dataset[2]. The biggest roadblocks at this time was the issue of **semantic ambiguity** [1].

In the modern day, we have developed new systems that have included depth data, and 3D point clouds, which are a series of points that are constructed in 3D, so that systems can understand what an object or a room is even if the lighting is uneven or captured at unconventional viewpoints as well as decrease the effect of semantic ambiguity[2]. These systems can be extremely accurate but complex and very heavy. These systems have several drawbacks. They are extremely expensive

to train and run, often requiring servers for computations. This means that no matter how accurate these models are, they suffer from not being able to be run on an autonomous unit. Taking into consideration these issues, we wanted to create our own system that addressed this, that was lightweight but also was robust against class imbalances from the data.

System Benchmarks and Practical Implementation:

Research comparing supervised learning algorithms says that while Neural Networks provide the highest top-1 accuracy, around ~77, they involve a long training process. They're also relatively fast, with the EMSANet framework, a single neural network model, achieves an inference throughput of 24.5 FPS on an NVIDIA Jetson AGX Xavier[3].

SVMs are recognized as highly robust and competitive with neural networks but are often slower during real-time inference. Naive Bayes is also identified as strong and a simpler alternative; Naive Bayes, in particular, shows only a 7% performance drop compared to neural networks while being significantly faster and easier to train[2].

Bottlenecks and Roadblocks:

Semantic Ambiguity, as mentioned earlier, is one roadblock for indoor scene recognition, for example similar arranged rooms like, libraries, archives, and bookstores often confuse models. Another thing that we have to assume when doing indoor scene recognition is the "**closed world assumption,**" where systems are trained on a fixed, predefined set of labels[1][4]. This makes them "fragile" in the wild as models can't reason about things they haven't seen. The "closed world assumption," for the scope of our project will just have to be upheld.

There is a tradeoff between achieving high accuracy and maintaining the computational efficiency required for mobile and robotic units. While enormous server-level models (e.g., ResNeXt with 88M parameters) achieve peak benchmarks, they are often too heavy for edge devices[2]. Practical implementation is restricted by the limited computing and battery capabilities of mobile systems, which require low-latency inference (e.g., under 100ms) to be useful for autonomous navigation[4].

References:

[1] D. Zeng et al., "Deep Learning for Scene Classification: A Survey," arXiv:2101.10531, 2021.

[2] B. Gamage, "An embarrassingly simple comparison of machine learning algorithms for indoor scene classification," arXiv:2109.12261, 2021.

[3] D. Seichter, S. B. Fischedick, M. Köhler, and H.-M. Groß, "Efficient Multi-Task RGB-D Scene Analysis for Indoor Environments," arXiv:2207.04526, 2022

[4] N. Patel, P. Krishnamurthy, and F. Khorrami, "RAZER: Robust Accelerated Zero-Shot 3D Open-Vocabulary Panoptic Reconstruction with Spatio-Temporal Aggregation," arXiv:2505.15373, 2025.

Colin Chu, Le Fan Fang

ML Methodology and Data Collection

Dataset

The dataset that we used was an [MIT Indoor Scene Recognition Database]. The dataset covers 67 unique indoor scene categories, such as libraries, restaurants, malls, bedrooms etc. We chose this dataset as it had a very wide variety of categories that we can predict, and also already had separate labeled folders ready for creating our models with minimal data cleaning.

Data Preprocessing

For the data preprocessing stage, we grouped the original dataset into 5 main categories: stores, homes, public spaces, leisure and working spaces. We did this because the dataset would be too small to split up into 67 categories. Even though we had 15620 images, if we split them up into 70 categories, we would have around 200-250 images for each category, which we decided wasn't enough data for a CNN model to converge. Finally to prepare for the CNN process, we turned the images to grayscale then into numpy arrays.

Model 1 - CNN

For the CNN model itself, we would input a grayscale 128x128 image, then pass it through 3 convolutional layers with ReLU activation functions, with max pooling between every layer. We then flattened the output of the convolutional layer, then passed it through 2 hidden layers with ReLU activations, first to 256 features then to 128 features. Finally, these features are then passed through a final softmax output layer to give us probabilities for each of the 5 categories. Our model was all trained using the ADAM optimizer through 20 epochs.

Initially, we found that our model was very quickly overfitting, with training accuracies of 99% at less than 10 epochs, however with a significantly lower test accuracy at 40%, even after implementing max pooling. We decided to add dropouts to each hidden layer of 0.5 initially, but then we still saw quick overfitting within 20 epochs with significantly lower test accuracies, so we decided to add dropout layers between every convolutional layer as well. We ended up with 5 total dropout layers, 3 after each convolutional layer of 0.25 and 2 after each hidden layer of 0.5.

Model 2 - Bayesian Logistic Regression with CNN features

For a second model, we decided to construct a Bayesian Logistic Regression model in R, but using the features trained using CNN in our first model. To prepare for the integration of the Bayesian model, we removed the softmax output layer and extracted the 128-dimensional feature embeddings to a CSV file for use in the Bayesian model. Since we used an isotropic Gaussian prior, we needed to normalize the extracted features before exporting it for our Bayesian model.

For the sampling process for Bayes, we manually implemented a Bayesian Metropolis-Hasting sampler, which was chosen over other samplers because our logistic likelihood has no closed-form posterior. While constructing this sampler, we ran into issues of probabilities between the five categories multiplying out to 0 when computing the joint distribution (derived from prior multiplied by likelihood). To get around this issue, we switched to a log prior and a log likelihood with an added smoothing term to avoid log(0) issues. Using a log prior and likelihood made it so that our joint was adding logs together rather than multiplying them. Due to this change, we accordingly had to calculate our acceptance ratio (alpha) by subtracting the log joint of the current weights from the log joint of the proposed weights.

With the log joint, we implemented our Metropolis sampler with the Gaussian prior centered at zero with a 25% burnout, and sampled independently for each of the 5 categories, creating a separate logistic classifier for each of the 5 categories, which would give us a posterior probability distribution of weights given an input of 128 feature embeddings and a corresponding labels for their category.

Summary

To summarize, we condensed our categories from 67 to 5 categories for a larger sample size per category, then we trained one CNN model based on labels of 5 categories, then used features extracted from this CNN model as inputs for a Bayesian Logistic model built on that as our second model. In addition to the bayesian and the CNN models, we attempted to build a second model that narrowed down the categories into room types, with the exact same structure as the initial CNN, except with the predictions being the probabilities of the room types for each of the 5 categories that we already predicted.

Colin Chu, Le Fan Fang

ML Results and Interpretation

CNN Model

While training our CNN model, we repeatedly tuned our CNN parameters to account for overfitting and underfitting, while keeping them reasonable. This included tuning the number of convolutional layers, number of hidden layers, activation functions, but as described earlier we spent the most time adding dropout layers to prevent over/underfitting.

Our resulting CNN model ended with a 74.5% training accuracy, with a 49.7% test accuracy after 20 epochs. Although we still have some overfitting, we didn't want to increase the dropout further, especially since we had 5 dropout layers already. We were already somewhat satisfied with this, since it already showed significant improvement than a model guessing at random or guessing only 1 category (around 20% accuracy), but our whole project goal was to combine two models to give predictive uncertainty on top of the classifier.

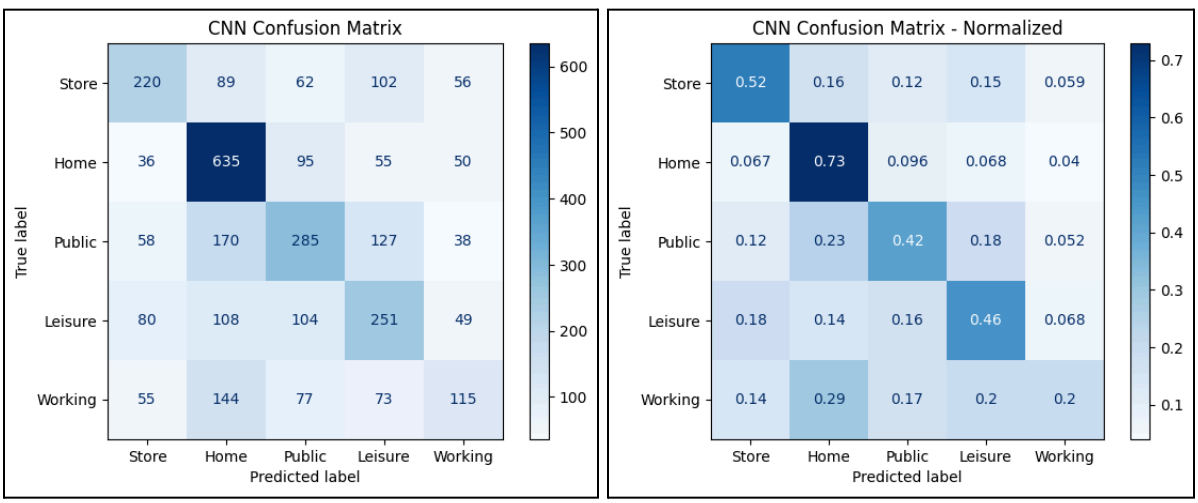

Figure 1-2: Confusion Matrices for our CNN Model (Raw Counts vs. Normalized)

Looking at the confusion matrix for this model, we can see that we are by far predicting the home category the most. However, we do see a heavy class imbalance, even though we stratified our data according to the original data, which reflects on the original dataset of images. Regardless, looking at the home column (predicted), it looks like for each true label (row) we're over predicting home, especially in public spaces and working spaces. This could make sense as stores and leisure spaces perhaps have more distinct elements between them compared to the other three categories.

Combined Model

For our combined Bayesian model with CNN features, we also repeatedly tuned our parameters for our sampler and our actual classifier model. Our original Bayesian model didn't improve on our

CNN model, with a similar accuracy of ~50%. To evaluate our Bayesian model, we looked at 4 metrics: trace plots, ACF plots and overall predictive accuracy across all categories.

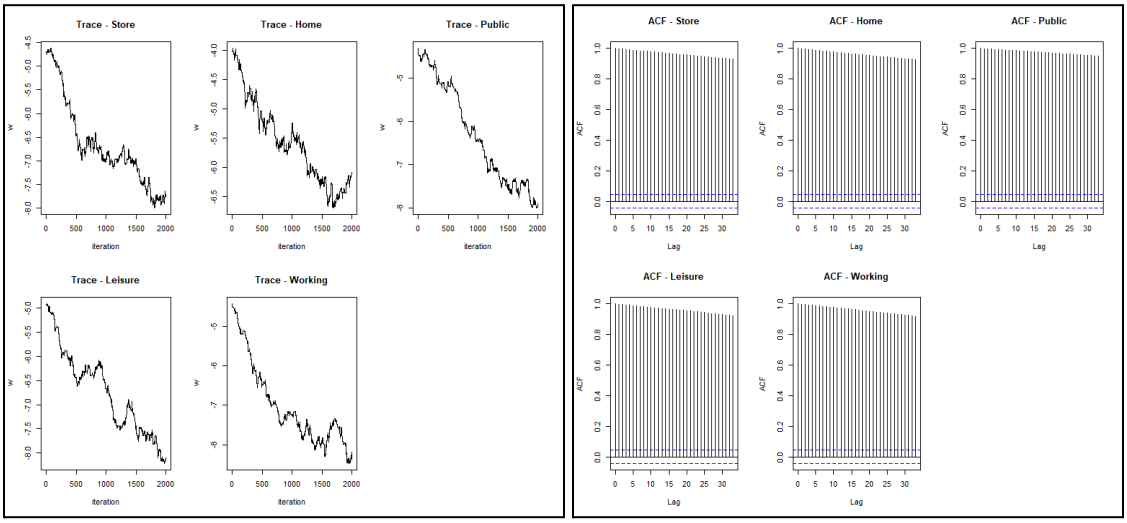

Figure 3-4: Trace Plots and ACF Plots for Each Category for the Final Bayesian Model

We quickly observe some obvious issues: our trace plots also show that all 5 chains show a downward trend, which means our chain has not yet converged. Our ACF plots show high autocorrelation, with an ACF of 0.9 even at lag 25. This meant that our samples were very correlated, and we had a very low amount of independent draws across our initial 2000 samples. We then attempted to thin out our samples, by increasing the number of samples we drew from our Metropolis sampler, but then taking every 10th sample. Although we did see minor improvement, with our ACF plot showing some decay, we decided to thin out our samples further.

The final version of our Bayesian logistic model thinned out every 25th sample from the 10000 samples. Although we are leaning on not having enough samples, this gave us the best model accuracy while still having enough samples for our model to predict from.

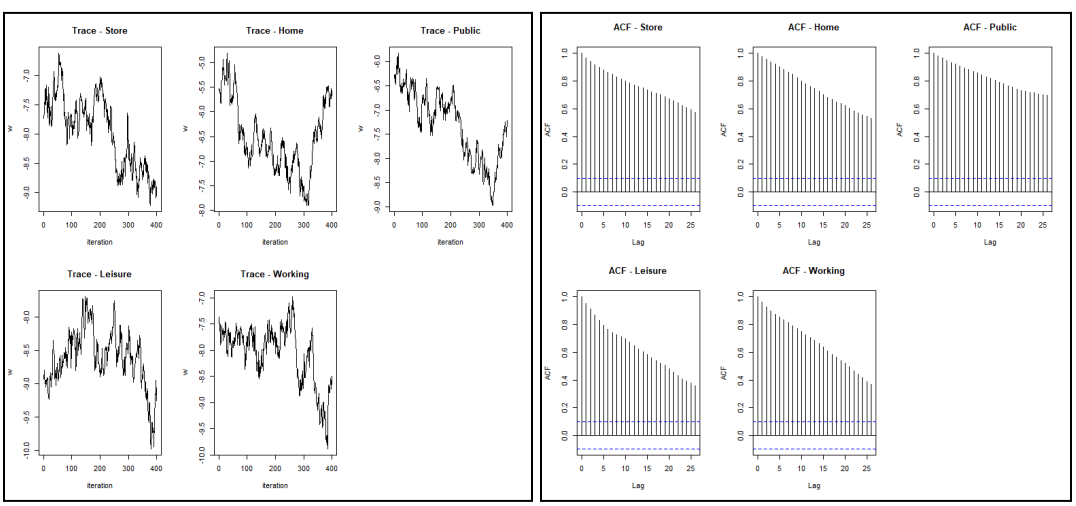

Figure 5-6: Trace Plots and ACF Plots for Each Category for the Final Bayesian Model

Our trace plots have improved after thinning samples, since they look more like random noise converging around a mean, however still not perfect. We also see a small improvement in our ACF plots, since they are now decaying slowly compared to not at all. Despite this, we see a sizable increase in predictive accuracy, from 49.7% to 63.1% accuracy for our Bayesian model.

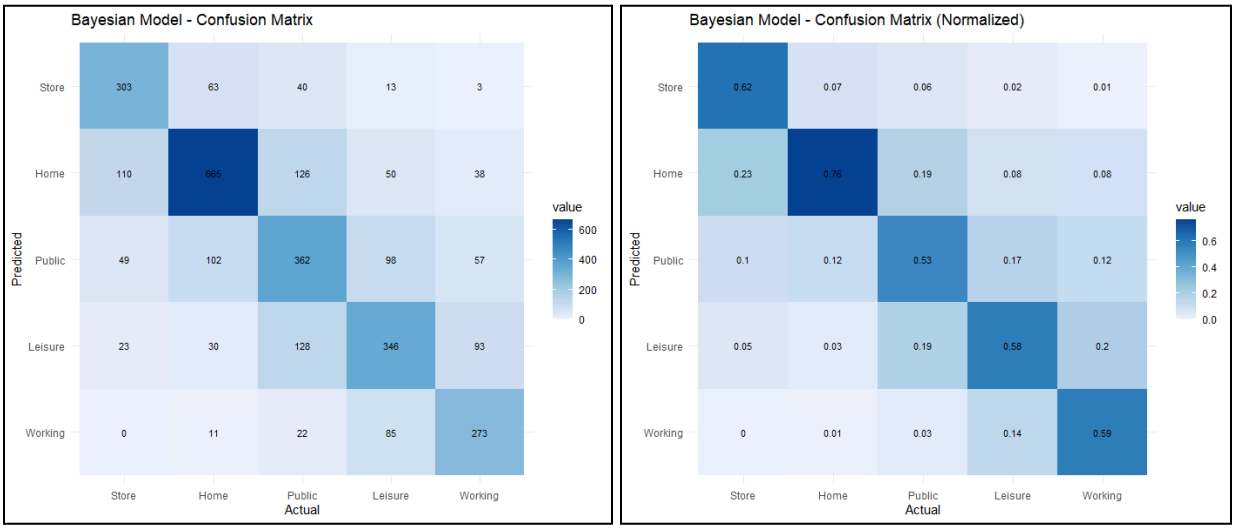

Figure 5-6: Trace Plots and ACF Plots for Each Category for the Final Bayesian Model

Looking at our confusion matrices, again our model is best at predicting the home category, but the other categories have caught up significantly (similar shade of blue in the matrix). Looking at the normalized confusion matrix, we also see less false predictions for home, which is visually shown by the home column having a lighter shade of blue than our confusion matrix for our CNN model. Our weakest performing class is public spaces, however all four categories besides home all have relatively similar predictive accuracies. Overall, our confusion matrix shows good results, our darkest cells are on the diagonal (correct predictions), and there are no near-zero diagonals, so our model has adapted its features to differentiate between each category individually.

Colin Chu, Le Fan Fang

Conclusions and Future Work

Our Bayesian model built on CNN features had a resulting accuracy of 63.1%, which we were very satisfied with considering the scope of our project. Due to constraints in our programs and costs, we had to scale to 128x128 grayscale images, and CNN isn't complex enough for object detection, which as a human is one of the main ways to differentiate between rooms/scenes.

Despite there not being that much data for each category, we still saw class imbalances in the confusion matrices. The distribution of images in the 5 categories were about the same, but some categories had a lot more room types. We believe that the "Home" category was predicted much better as there's less room types under that category, which means that images per room rate was higher. Our two worst categories, public spaces and working spaces, were predicted not as well as the home category. We also couldn't resolve the high autocorrelation in our manual MCMC chains even after thinning, most likely due to our feature space having 128 features, and our output being between 5 categories, which makes it hard to implement manually.

Overall, we would say that this project was a success. We showed that a Bayesian approach that gives uncertainty distributions was somewhat valuable, as it made our initial feature extraction model much better. This is especially important for scene recognition where prediction confidence also matters.

In the future, if we ever get more computation power, we would love to train more complex models that can predict more specific categories, and see if the bayesian modelling in conjunction with CNN feature extraction, can make an accurate predictor for Indoor Scene Recognition. We would also like to work with an even bigger dataset, while 15,000 images is a lot, each room type typically has 100-200 images, which is almost impossible to train a model that converges for each room type.

Colin Chu, Le Fan Fang

Appendix: Implementation

CNN Model

```
#input layer
inpx = Input(shape=(128, 128, 1))

#conv layer 1 + pooling + dropout
con_layer = Conv2D(32, kernel_size=(3,3), activation='relu',
padding='same')(inpx)
pool_layer = MaxPooling2D(pool_size=(2,2))(con_layer)
drop_conv1 = Dropout(0.25)(pool_layer)

#conv layer 2 + pooling + dropout
con_layer2 = Conv2D(64, kernel_size=(3,3), activation='relu',
padding='same')(drop_conv1)
pool_layer2 = MaxPooling2D(pool_size=(2,2))(con_layer2)
drop_conv2 = Dropout(0.25)(pool_layer2)

#conv layer 3 + pooling + dropout
con_layer3 = Conv2D(128, kernel_size=(3,3), activation='relu',
padding='same')(drop_conv2)
pool_layer3 = MaxPooling2D(pool_size=(2,2))(con_layer3)
drop_conv3 = Dropout(0.25)(pool_layer3)

#flatten for hidden layer
flat_G = Flatten()(drop_conv3)

#hidden ReLU 1 + dropout
hid_layer = Dense(256, activation='relu')(flat_G)
dropout = Dropout(0.5)(hid_layer)

#hidden ReLU 2 + dropout
hid_layer2 = Dense(128, activation='relu')(dropout)
dropout2 = Dropout(0.5)(hid_layer2)

#softmax output for 5 categories
out_layer = Dense(5, activation='softmax')(dropout2)
```

Colin Chu, Le Fan Fang

Bayesian Model + Metropolis-Hasting Sampler

```
# Metropolis sampler
metropolis_bayesian_logistic <- function(X, y, n_features,
                                n = no_samples, sigma = 0.05, burn = 0.25) {

  total_samples <- as.integer(n / (1 - burn))
  m <- as.integer(total_samples * burn)  # burn-in amount
  accepted <- 0

  samples <- matrix(0, nrow = total_samples, ncol = n_features)
  w_tilde <- matrix(0, nrow = n_features, ncol = 1)

  for (i in 1:total_samples) {
    w_proposal <- w_tilde + matrix(rnorm(n_features, 0, sigma), ncol = 1)

    # Acceptance ratio (subtract logs since we're using a log joint)
    alpha <- p(w_proposal, X, y) - p(w_tilde, X, y)

    if (log(runif(1)) < alpha) {
      w_tilde <- w_proposal
      accepted <- accepted + 1
    }

    # Store sample
    samples[i, ] <- w_tilde
  }

  # Acceptance rate (ideally around 23.4%)
  acceptance_rate <- accepted / total_samples
  cat(sprintf("Acceptance rate: %.3f\n", acceptance_rate))

  # Discard burn-in
  samples <- samples[(m + 1):nrow(samples), ]
  return(samples)
}

class_names <- c("Store", "Home", "Public", "Leisure", "Working")

# Outer loop: one binary classifier per super-category
for (class_id in 0:(n_classes - 1)) {
  cat(sprintf("Fitting class %d/5: %s\n", class_id + 1, class_names[class_id +
1]))

  y <- as.numeric(y_train == class_id)
  all_w_samples[[class_id + 1]] <- metropolis_bayesian_logistic(X_train, y,
n_features)
}
```

Github Link: https://github.com/EthanTheFanger/IndoorSceneRecognition